In [1]:
```
# a. Import necessary packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense
from sklearn.model_selection import train_test_split

# b. Load and preprocess the image data
data = pd.read_csv('cifar-10.csv')

# Separate labels and pixel values
X = data.drop('label', axis=1).values
y = data['label'].values

# Normalize pixel values
X = X.astype('float32') / 255.0

# Reshape images into 32x32 RGB format
X = X.reshape(-1, 32, 32, 3)

# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# c. Define the CNN architecture
model = Sequential([
    Input(shape=(32, 32, 3)),

    Conv2D(32, (3, 3), activation='relu', padding='same'),
    MaxPooling2D((2, 2)),

    Conv2D(64, (3, 3), activation='relu', padding='same'),
    MaxPooling2D((2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# d. Train the model
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.1
)
```

```
# e. Estimate model performance
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")

# Display sample prediction
prediction = model.predict(X_test[:1])
print("Predicted Class:", np.argmax(prediction[0]))
print("Actual Class:", y_test[0])
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Pleas
e use WSL2 or the TensorFlow-DirectML plugin.
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
dense (Dense)	(None, 128)	524,416
dense_1 (Dense)	(None, 10)	1,290

Total params: 545,098 (2.08 MB)

Trainable params: 545,098 (2.08 MB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
563/563 ━━━━━━━━━━━━━━━━━━━━ 18s 26ms/step - accuracy: 0.4162 - loss: 1.6359 - va
l_accuracy: 0.4970 - val_loss: 1.4044
Epoch 2/10
563/563 ━━━━━━━━━━━━━━━━━━━━ 14s 25ms/step - accuracy: 0.5351 - loss: 1.3216 - va
l_accuracy: 0.5518 - val_loss: 1.2761
Epoch 3/10
563/563 ━━━━━━━━━━━━━━━━━━━━ 14s 25ms/step - accuracy: 0.5847 - loss: 1.1823 - va
l_accuracy: 0.5835 - val_loss: 1.1768
Epoch 4/10
563/563 ━━━━━━━━━━━━━━━━━━━━ 14s 25ms/step - accuracy: 0.6172 - loss: 1.0861 - va
l_accuracy: 0.5807 - val_loss: 1.2334
Epoch 5/10
563/563 ━━━━━━━━━━━━━━━━━━━━ 16s 28ms/step - accuracy: 0.6474 - loss: 1.0079 - va
l_accuracy: 0.5987 - val_loss: 1.1388
Epoch 6/10
563/563 ━━━━━━━━━━━━━━━━━━━━ 15s 27ms/step - accuracy: 0.6746 - loss: 0.9318 - va
l_accuracy: 0.5997 - val_loss: 1.1446
Epoch 7/10
563/563 ━━━━━━━━━━━━━━━━━━━━ 15s 27ms/step - accuracy: 0.6971 - loss: 0.8646 - va
l_accuracy: 0.6127 - val_loss: 1.1021
Epoch 8/10
563/563 ━━━━━━━━━━━━━━━━━━━━ 13s 23ms/step - accuracy: 0.7155 - loss: 0.8141 - va
l_accuracy: 0.6355 - val_loss: 1.0877
Epoch 9/10
563/563 ━━━━━━━━━━━━━━━━━━━━ 14s 26ms/step - accuracy: 0.7369 - loss: 0.7518 - va
l_accuracy: 0.6252 - val_loss: 1.0978
Epoch 10/10
563/563 ━━━━━━━━━━━━━━━━━━━━ 15s 26ms/step - accuracy: 0.7577 - loss: 0.6981 - va
l_accuracy: 0.6265 - val_loss: 1.1272
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.6318 - loss: 1.1134
Test Loss: 1.1134032011032104
Test Accuracy: 63.179999589920044 %
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 199ms/step
Predicted Class: 7
Actual Class: 7
```

In [ ]: